

CS 630: Software Testing and Quality Assurance

5-2 Paper: System-Wide Integration and Regression Testing Plan

Malgorzata Debska

Southern New Hampshire University

Dr. Ben Torrales

August 5, 2026

System-Wide Integration and Regression Testing Plan

LogiCo's enterprise software system supports several business functions, including user management, route optimization, package tracking, billing, and third-party services such as Google Maps, FedEx, and Stripe. The company has moved from a monolithic application to a combination of monolith and microservices, but the services still share the same Oracle database and use non-versioned APIs. As a result, even a small update can affect other parts of the system. This recently occurred when a billing update caused problems in the route optimization module. To prevent similar issues, LogiCo needs a structured integration and regression testing process that detects defects before software reaches production.

Designing a System-Wide Integration Testing Process

Requirements Definition

Integration testing should begin during the requirements phase. Before development starts, the team should identify every module affected by the change and document how those modules interact. For example, changes to the billing service should also be evaluated for their impact on package tracking, route optimization, and payment processing. Identifying these dependencies early makes it easier to build effective integration tests (Pressman & Maxim, 2020).

Planning

During sprint planning, the team should determine which integration tests are required for the new feature. Since LogiCo supports Web, Android, and iOS applications, testing should include

all three platforms. The team should also identify any third-party APIs that need to be validated before deployment.

Coding

Developers should create automated integration tests while writing new code. These tests should verify that services continue exchanging data correctly after changes are made. Since LogiCo does not use versioned APIs, automated testing is essential for identifying problems before they affect other systems (Pressman & Maxim, 2020).

Programmer-Facing Testing

Before code is merged into the main branch, developers should run unit, integration, and API tests. If any test fails, the code should be corrected before approval. Following the same testing process across all teams improves consistency and prevents unstable code from reaching production.

Customer-Facing Testing

After developer testing is complete, the application should be deployed to the staging environment for end-to-end testing. Testers should verify key business functions, including logging in, creating shipments, calculating delivery routes, tracking packages, and processing payments on the Web, Android, and iOS applications. Improving the reliability of the staging environment will also reduce testing delays.

Continuous Integration

Every code change should automatically trigger the Jenkins pipeline. Jenkins should execute unit, integration, and API tests before deployment. Developers should not bypass these steps, even when working under tight deadlines. Automated testing provides faster feedback and reduces the chance of production failures.

Deployment

Once testing is complete, the software can be deployed. Billing services should continue requiring manual approval because they process regulated information, while other services can use gradual deployments to reduce the impact of unexpected problems (Pressman & Maxim, 2020).

Production Monitoring

Testing should continue after deployment. Developers should monitor application logs, API performance, system errors, and customer reports. If problems appear after a release, alerts should be generated immediately, so issues can be resolved quickly and used to improve future testing.

Regression Testing Plan

Regression testing should run after every code change, every pull request, and before every production deployment. The regression suite should include user login, account management, route optimization, package tracking, billing, database operations, communication between services, third-party API integrations, and testing on the Web, Android, and iOS platforms.

Whenever a production defect is discovered, a new regression test should be added so the same problem does not occur again. This strengthens the regression suite over time and improves software quality (Pressman & Maxim, 2020).

Addressing Common Failure Points

This testing strategy addresses several issues identified in the LogiCo scenario. Automated integration testing helps detect problems caused by non-versioned APIs before they affect connected services. Regression testing ensures existing functionality is not broken by new updates. Testing the shared Oracle database helps identify data-related issues, while cross-platform testing confirms that updates work correctly on the Web, Android, and iOS applications. Using one standardized testing process also improves consistency across development teams.

Supporting Long-Term System Stability and Reliability

This process improves long-term stability by identifying problems throughout development instead of after deployment. Automated testing provides immediate feedback, allowing developers to fix issues before they reach production. As new regression tests are added over time, the software becomes more reliable because previously fixed defects continue to be tested with every release. This approach reduces customer-facing bugs and increases confidence in future updates (Pressman & Maxim, 2020).

Adapting the Strategy to Other Enterprise Applications

This testing strategy can also be used by organizations in healthcare, banking, retail, insurance, and manufacturing. These industries also rely on multiple connected systems, databases, and external APIs. Integration testing, regression testing, continuous integration, and production monitoring improve software quality regardless of the industry because they ensure systems continue working together as new features are introduced.

References

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.